**Future Interns**

**Cyber Security Task 3 (2026)**

# API Security Risk Analysis Report

JSONPlaceholder — Public API Security Assessment

Prepared by Ebasa Getu

---

## Risk Summary

| # | Risk | Severity | OWASP | Business Impact |
|---|---|---|---|---|
| 1 | Missing Authentication | High | API2 | No ability to track or audit API usage. Impossible to enforce access controls. A... |
| 2 | Excessive Data Exposure | High | API3 | PII (email, phone, home address, geolocation) is exposed unnecessarily. This con... |
| 3 | Broken Object Level Authorization (IDOR) | High | API1 | Complete breakdown of multi-tenant data isolation. In a SaaS application, Tenant... |
| 4 | Weak Rate Limiting | Medium | API8 | Brute-force attacks against endpoints (credential guessing, ID enumeration) beco... |
| 5 | Missing Input Validation | Medium | API9 | SQL/NoSQL injection vulnerabilities if database queries are dynamically construc... |
| 6 | Sensitive Data in URL Parameters | Low | API3 | User IDs and filtering criteria logged in server access logs and CDN logs. Analy... |
| 7 | Permissive CORS Configuration | Low | API7 | Any malicious website can make credentialed API requests on behalf of users if c... |

# Finding 1: Missing Authentication

| Severity | High |
|---|---|
| OWASP | API2 |
| Endpoint | /* (all endpoints) |

## Description

The API requires no authentication whatsoever. Any client can access all resources without providing an API key, token, or credentials. Even a fabricated Bearer token is accepted without validation, meaning there is zero access control at the transport layer.

## Business Impact

No ability to track or audit API usage. Impossible to enforce access controls. Any third party can consume API resources. No accountability for data access. In a production SaaS environment, this would allow unlimited data scraping and abuse.

## Remediation

1. Implement API key authentication via X-API-Key header for machine-to-machine access
2. Deploy OAuth 2.0 / JWT for user-level authentication with short-lived tokens
3. Validate tokens on every single request via middleware
4. Reject unauthenticated requests with 401 Unauthorized and descriptive error codes
5. Log all authenticated requests with user identity for audit trails

## Evidence

```
# No auth header -> 200 OK
curl -s -o /dev/null -w "%{http_code}"
https://jsonplaceholder.typicode.com/posts
200

# Fake Bearer token -> also 200 OK
curl -s -o /dev/null -w "%{http_code}" -H "Authorization: Bearer faketoken123"
https://jsonplaceholder.typicode.com/posts/1
200
```

# Finding 2: Excessive Data Exposure

| Severity | High |
|---|---|
| OWASP | API3 |
| Endpoint | GET /users/{id} |

## Description

The /users endpoint returns far more data than any typical client needs, including email addresses, phone numbers, full physical addresses (street, suite, city, zipcode, and precise GPS coordinates), and company details. In a production blog or todo app, only a display name and avatar URL should be exposed.

## Business Impact

PII (email, phone, home address, geolocation) is exposed unnecessarily. This constitutes a GDPR/CCPA compliance risk. Attackers can harvest contact data for phishing campaigns. Geolocation data poses physical privacy risks.

## Remediation

1. Implement field-level filtering — only return fields the client explicitly requests via query parameters
2. Use GraphQL or JSON:API with sparse fieldsets for granular control
3. Create separate response DTOs per client type (public, authenticated, admin)
4. Never serialize internal database entities directly to API responses
5. Conduct regular response payload audits with automated schema validation

## Evidence

```
curl -s https://jsonplaceholder.typicode.com/users/1 | jq 'keys'
[
  "id", "name", "username", "email",
  "address", "phone", "website", "company"
]

address contains: street, suite, city, zipcode, geo (lat/lng)
This exposes geolocation for every user — unnecessary
```

# Finding 3: Broken Object Level Authorization (IDOR)

| **Severity** | **High** |
|---|---|
| OWASP | API1 |
| Endpoint | GET /posts?userId={id}, GET /users/{id} |

## Description

There is no authorization check on any resource. Any user can access any other user's posts, todos, or personal information simply by changing query parameters or URL segments. Sequential integer IDs make enumeration trivial.

## Business Impact

Complete breakdown of multi-tenant data isolation. In a SaaS application, Tenant A can read Tenant B's private data — financial records, medical history, private messages. Regulatory fines and irreparable reputational damage would follow.

## Remediation

1. Implement object-level authorization middleware on every resource endpoint
2. Compare requested userId against authenticated user identity server-side — never trust client input
3. Use UUIDs instead of sequential integers to reduce enumeration (defense-in-depth, not a replacement for auth)
4. Apply the principle of least privilege — users should only access resources they own
5. Conduct automated authorization fuzzing in CI/CD pipeline

## Evidence

```
curl -s "https://jsonplaceholder.typicode.com/posts?userId=1" | jq length
10
curl -s "https://jsonplaceholder.typicode.com/posts?userId=2" | jq length
10

Both return data — no authorization barrier between users
```

# Finding 4: Weak Rate Limiting

| **Severity** | **Medium** |
|---|---|
| OWASP | API8 |
| Endpoint | GET /posts |

## Description

While a rate limit header exists (1000 requests/hour), the limit is generous and there is no apparent throttling on rapid burst requests. Five sequential requests with zero delay all succeeded without a single 429 response.

## Business Impact

Brute-force attacks against endpoints (credential guessing, ID enumeration) become feasible. Resource exhaustion can degrade performance for legitimate users. Infrastructure costs from abuse scale linearly with attacker volume.

## Remediation

1. Implement per-IP and per-user rate limits with different tiers
2. Add exponential backoff with proper Retry-After headers on 429 responses
3. Use a sliding window algorithm instead of fixed window for smoother enforcement
4. Set up real-time alerting on anomalous request patterns
5. Deploy CAPTCHA or proof-of-work challenges for sensitive operations

## Evidence

```
for i in 1 2 3 4 5; do
  curl -s -o /dev/null -w "Request $i: %{http_code}\"
    https://jsonplaceholder.typicode.com/posts
done
Request 1: 200
Request 2: 200
...
Request 5: 200

Rate limit headers:
  x-ratelimit-limit: 1000
  x-ratelimit-remaining: 999
```

# Finding 5: Missing Input Validation

| **Severity** | **Medium** |
|---|---|
| OWASP | API9 |
| Endpoint | POST /posts |

## Description

The API accepts arbitrary JSON payloads without apparent validation. There is no schema enforcement, type checking, length validation, or content sanitization visible from responses.

## Business Impact

SQL/NoSQL injection vulnerabilities if database queries are dynamically constructed. Stored XSS if response data is rendered in web clients without sanitization. Data corruption from malformed payloads.

## Remediation

1. Define strict JSON Schema validators for all request bodies and enforce server-side
2. Validate input types, string lengths, numeric ranges, and allowed enum values
3. Sanitize all input before persistence — strip HTML, escape special characters
4. Use parameterized queries or ORM abstraction layers to prevent injection
5. Return descriptive validation error messages without leaking implementation details

## Evidence

```
curl -X POST https://jsonplaceholder.typicode.com/posts \
  -H "Content-Type: application/json" \
  -d '{"title":"foo","body":"bar","userId":1}'

Response: 201 Created — accepted without validation
```

# Finding 6: Sensitive Data in URL Parameters

| **Severity** | **Low** |
|---|---|
| OWASP | API3 |
| Endpoint | GET /todos?userId={id} |

## Description

User identifiers are passed as query string parameters, which are logged in plaintext by CDNs, reverse proxies, load balancers, and analytics tools. Sequential integer IDs also enable trivial user enumeration.

## Business Impact

User IDs and filtering criteria logged in server access logs and CDN logs. Analytics tools may inadvertently collect behavioral data tied to user identifiers. Easier reconnaissance for attackers mapping user base.

## Remediation

1. Use POST with body parameters for sensitive filter criteria
2. Consider adopting non-sequential identifiers (UUID v4) for user-facing resources
3. Implement access log sanitization to redact query parameters containing identifiers
4. Use HTTP/2 or encrypt query strings where supported

## Evidence

```
curl -s "https://jsonplaceholder.typicode.com/todos?userId=1"

userId=1 visible in URL — logged by every intermediary
```

# Finding 7: Permissive CORS Configuration

| Severity | Low |
|---|---|
| OWASP | API7 |
| Endpoint | All endpoints |

## Description

The API includes access-control-allow-credentials: true and broad CORS headers, allowing any website to make credentialed requests.

## Business Impact

Any malicious website can make credentialed API requests on behalf of users if cookies or browser-stored credentials are used. Cross-Site Request Forgery (CSRF) attacks become feasible.

## Remediation

1. Restrict Access-Control-Allow-Origin to a specific allowlist of trusted domains
2. Disable Access-Control-Allow-Credentials unless the specific endpoint requires it
3. Implement CSRF tokens for all state-changing operations
4. Use SameSite=Strict or SameSite=Lax cookie attributes

## Evidence

```
Response header:
  access-control-allow-credentials: true

Any origin can make credentialed requests — no origin validation
```